67-404 Blockchain Applications — Semester Project (ALL-IN-ONE)

This integrated handout merges the original sprint schedule, ICS policy, detailed rubrics, and submission instructions with the low-burden Metrics Micro-Pack. It includes an appendix with quick commands and templates.

0) At-a-glance (what's due when, and worth how much)

Week(s)	Milestone	Deliverables (highlights)	Weight
6–7	Sprint 1 – Problem & Blockchain Fit	1-pager concept; WhyChain Canvas; mini literature scan; team charter; low-fi UX; 2-min pitch	6%
8–9	Sprint 2 – Architecture & PoC	System diagram + threat model; contract/API skeletons; PoC (local/testnet); test plan; short dry-run demo	7%
10–11	Sprint 3 – MVP & UX	Testnet MVP; operable interface (CLI/Etherscan or thin GUI); tests + coverage; gas/fee baseline; usability test #1; Metrics Micro-Pack (v1)	9%
12–13	Sprint 4 – Beta & Audit	Beta on testnet; security + gas review; docs/runbook; deployment reliability; Slither summary; before/after gas table; light UX polish (basic empty/error states)	6%
14	Final Presentation	Slides + live demo + Q&A (9–10 min)	4%
14 (end)	Walkthrough Recording (~5 min)	Scenario-driven video with captions	2%
14 (end)	Final PDF Report	Lit review, rationale, design, code refs, evaluation, AI log, individual contribs, reflection	6%
—	Total	—	40%

How individual effort is graded: Each sprint grade = 80% group work + 20% individual contribution (ICS). See Section 7 for formula and examples.

1) Project purpose & learning goals

- Pick a problem that credibly needs blockchain; justify decentralization (not cargo-culting).
- Design secure, efficient smart contracts with clean interfaces and events.
- Deliver a usable dApp (wallet flows, error states, clarity).

    - Provide an operable interface for the core on-chain flow (CLI/Etherscan runbook or thin GUI).
- Measure correctness (tests), efficiency (gas/fees), and usability (user feedback).
- Work in sprints with professional practice, ethics, and transparent AI use.

2) Allowed scope, constraints, and tech stack

Minimum (baseline to pass):

- 1–3 smart contracts on an EVM-compatible chain (local or testnet).
- An operable interface for the core on-chain flow: CLI scripts (Foundry/Hardhat), an Etherscan runbook, or a thin React dApp GUI is optional.
- A minimal data layer (events or off-chain store; IPFS optional).
- Automated tests with coverage for core logic.

Recommended low-friction tools (choose what fits; full steps are in the Project Playbook and Glue Labs):

- Contracts: Foundry (default) or Hardhat; OpenZeppelin libraries; Remix for quick experiments.
- Client (only if you choose a GUI): React/Next.js + wagmi + viem; simple design system if desired.
- Storage (optional): IPFS via a pinning service; or a simple DB for off-chain data.
- Analysis (encouraged): Slither/static analysis; coverage; gas reporting.
- Indexing (optional): The Graph or a lightweight event listener.

Use public testnets or local chains only — no real funds, no token sales, no inducements.

3) Idea quality model (grading lenses)

- Desirability — stakeholder needs and scenarios.
- Decentralization Fit — shared state across distrustful parties; auditability; programmable assets; verifiable credentials; composability.
- Feasibility — can we build it this semester?
- Viability — costs, risks, adoption path.

4) What's in/out for topics (examples)

- Promising: verifiable credentials; DAO-lite governance; supply chain provenance; ticketing with transfer rules; micropatronage/royalties; ERC-4337 experiments; toy-scale ZK flows.
- Too trivial: hello-world ERC-20 with no mechanism; basic NFT minter without policy/UX; pure client demo without contracts.

5) Sprint schedule & deliverables

Sprint 1 (Weeks 6–7): Problem & Blockchain Fit — 6%

- Concept 1-pager (problem, users, target outcomes).
- WhyChain Canvas (stakeholders, trust boundaries, on/off-chain split, risks).
- Mini literature scan (5–8 sources; what exists; your differentiator).
- Low-fi UX (flows + key wireframes).
  - State machine & events (contract-level states, invariants, and event schema)
- Team charter (roles, decision process, definition of done).
- 2-minute pitch during check-in.
- Check-in with instructor/TA (10–15 min). Due: Week 7, Thursday, 11:59pm.

Sprint 2 (Weeks 8–9): Architecture & PoC — 7%

- Architecture diagram (contracts, events, client, off-chain components).
- Threat model (roles/permissions, trust assumptions, obvious attacks).
- Contract/API skeletons (interfaces, state, events).
- PoC running locally or on a testnet (happy-path).
- Test plan + initial tests (unit/property).
- Check-in + short dry-run demo. Due: Week 9, Thursday, 11:59pm. PQR1 due.

Sprint 3 (Weeks 10–11): MVP & UX — 9%

- MVP on testnet (core features complete).
- Operable interface (CLI/Etherscan or thin GUI) that completes the core flow; show pending→confirmed/failed states; link to Etherscan for txs.
- Test suite with evidence of coverage for core logic.
- Gas baseline for key functions (document method).
- Usability Test #1 (≥3 testers; notes & changes).
- Metrics Micro-Pack (v1): gas-snapshot.txt (or Hardhat gas report), coverage-summary.txt (or screenshot), latency.csv (three end-to-end runs of your primary write on Sepolia: latencyMs, gasUsed, effectiveGasPrice, txHash, blockNumber, calldataBytes, status), deploy.json (chain, address, creation tx, deploy block, compiler, Etherscan link).
- Check-in + live demo. Due: Week 11, Thursday, 11:59pm.

Sprint 4 (Weeks 12–13): Beta & Audit — 6%

- Beta on testnet with improved UX and edge-case handling.
- Security pass: Slither summary (counts by severity + accepted-risk notes).
- Gas optimization report: before/after table and trade-offs.
- Deployment & reliability: idempotent scripts; repeatable deploys; tags.
- Developer docs/runbook (setup, deploy scripts, test commands).
- Light UX polish (basic empty/error states only).

- Check-in + release plan for Week 14. Due: Week 13, Thursday, 11:59pm.

Week 14: Finals

- Group presentation (9–10 min + Q&A).
- Walkthrough recording (~5 min, captioned).
- Final PDF report (see Section 6).
- Code freeze and repository link. PQR2 due.

6) Final deliverables details

A. Presentation (9–10 min): Problem → Why blockchain → Architecture → Demo (2–4 min) → Evidence (tests, gas, usability) → Lessons & future work. Every member speaks.

B. Walkthrough video (~5 min): Scenario end-to-end, then 30–60 seconds on architecture/code hotspots. Include captions and visible commit hash/tag.

C. Final PDF report (8–12 pages, excluding appendix):

- Abstract; Introduction; Literature review; Design rationale & WhyChain; Architecture & data; Implementation.
- Evaluation: tests, coverage, gas/efficiency, usability findings, limitations.
- Ethics & risk; AI usage log & reflection; Individual contributions; References.
- 1-page Metrics Micro-Pack summary — a small table aggregating gas, latency, fees, and coverage, plus 2–3 bullet insights.

7) Individual contribution & catching freeriders (ICS)

Per sprint, each student receives: Student Sprint Grade = Group Score × (0.8 + 0.2 × ICS), where ICS ∈ [0.50, 1.15].

ICS components (each sprint):

- Peer evaluations — Peer Quick Review (PQR) (50%) — confidential 1–5 ratings + comments. Mapping: 5→1.15, 4→1.05, 3→0.95, 2→0.75, 1→0.50.
- Evidence of work (35%) — commits/PRs/issues, docs, test artifacts, design work; quality over raw counts. Mapping: Strong→1.10, Moderate→1.00, Weak→0.80, Minimal→0.60.
- Professionalism (15%) — on-time check-ins, responsiveness, constructive teamwork. Mapping: Strong→1.05, Moderate→1.00, Weak→0.85.

Early-warning policy: If ICS < 0.70 in Sprint 2 or Sprint 3, we schedule a required 1:1 and set an improvement plan. If ICS < 0.60 twice, the student may receive zero on the 20% individual portion for those sprints and may be excluded from presentation speaking roles.

8) Work ethics, AI policy, safety & accessibility

- Academic integrity: Write your own code/designs; cite licensed code you borrow; no reuse of prior cohorts.
- AI use (allowed with transparency): brainstorming, code suggestions/snippets, refactoring hints, doc/help-text drafts, test scaffolding, diagram suggestions. Log significant usage with prompt + summary; review and test all outputs.
- Prohibited: submitting AI-generated work without review; fabricating citations/data; using AI to write peer reviews; exposing secrets/keys. Cite models/tools (e.g., model name + date).
- Security & safety: testnets/local only; never real funds; no personal/sensitive data on-chain; disclose known limitations; never attack others' systems.
- Accessibility & inclusion: captions, readable contrast, keyboard-navigable UI.
- **Non-Compliance and Deductions**
  Failure to comply with any policy, requirement, or ethical guideline stated in this document or in the course syllabus may result in a grade deduction or other penalty. The **teaching team reserves full discretion** to determine the extent of any deduction based on the **magnitude and impact** of the issue. Repeated or serious violations may also be referred to institutional academic integrity channels.

9) Rubrics

Shared performance levels: Excellent (A), Good (B), Developing (C), Insufficient (D/F).

Sprint 1 – Problem & Blockchain Fit (100 pts → 6%)

Criterion	Pts	Excellent → Insufficient (signals)
Problem statement & stakeholder needs	20	Clear user pain, concrete scenarios, measurable outcomes ↔ vague/assumed needs
Decentralization fit ("Why blockchain?")	25	Trust boundaries; on/off-chain split; alternatives considered ↔ hand-wavy
Value proposition & adoption path	15	Credible benefits; risks & mitigations ↔ unclear value
Feasibility & risk scan	15	Realistic scope; constraints recognized ↔ ignores constraints
Low-fi UX & flows	10	Key flows; error/edge states ↔ happy-path only
Literature scan	10	5–8 relevant sources; differentiator ↔ thin/unrelated

Team charter & professionalism	5	Roles, norms, DoD, timelines; on-time check-in ↔ missing/late

Sprint 2 – Architecture & PoC (100 pts → 7%)

Criterion	Pts	Excellent → Insufficient
System architecture diagram	20	Precise components/interfaces; events & data flow ↔ unclear boxes/arrows
Threat model & permissions	15	Roles, invariants, failure modes ↔ superficial
Contract/API skeletons	20	Clean interfaces, modifiers, events; compilable ↔ incoherent
Running PoC (local/testnet)	25	Core logic demo; meaningful logs/events ↔ nonfunctional
Test plan & initial tests	10	Targets invariants & edge cases ↔ ad-hoc
Project management & velocity	10	Realistic backlog; burndown ↔ chaos

Sprint 3 – MVP & Measurements (100 pts → 9%)

Criterion	Pts	Excellent → Insufficient
MVP completeness vs. scope	30	Core user story fully works on testnet ↔ gaps block core tasks
~~**Frontend integration & UX**~~	~~15~~	~~Clear states; errors handled; smooth wallet flows ↔ confusing UX~~
Test suite depth & coverage	20	Unit/property tests; coverage evidence; negatives ↔ shallow
Security hygiene & code quality	20	CEI; access control; lint/CI ↔ risky patterns
Gas latency measurement (Micro-Pack v1)	15	Key funcs measured; latency.csv & deploy.json present ↔ absent/dubious
~~**Usability test #1 & iteration**~~	~~10~~	~~≥3 testers; changes implemented ↔ token effort~~
Interface & ops reliability (GUI or CLI/Etherscan)	5	Clear pending/confirmed/failed; Etherscan links ↔ opaque
Reproducibility (RUN.md, scripts)	5	Clone→install→run core flow; deterministic steps ↔ brittle
Professionalism	5	Prepared, time-boxed,

(check-in)	action items ↔ unprepared

Sprint 4 – Beta & Audit (100 pts → 6%)

Criterion	Pts	Excellent → Insufficient
Feature completeness vs. plan	20	All planned features; justified descopes ↔ unfinished core
Security review & fixes	25	Static + manual review; issues resolved ↔ ignored warnings
Gas optimization & cost analysis	20	Before/after data; trade-offs explained ↔ hand-wavy
Deployment & reliability	15	Repeatable scripts; idempotent deploy; tags ↔ brittle
Documentation & runbook	10	Setup, deploy, env, tests; clear ↔ missing steps
Usability polish & edge cases	5	Basic empty/error states, latency handling ↔ rough edges
Professionalism (check-in & release plan)	5	Stable demo plan; crisp narrative ↔ flaky/unclear

Final Presentation (100 pts → 4%)

Criterion	Pts
Problem framing & stakes	15
Why blockchain + architecture clarity	20
Live demo (reliable, relevant)	25
Evidence: tests, gas, usability, limitations	20
Delivery: structure, timing, visuals, Q&A	20

Walkthrough Recording (100 pts → 2%)

Criterion	Pts
Scenario-driven flow (start→finish)	30
Architecture & code highlights	25
Repro steps (env, accounts, scripts)	20
A/V quality & captions	10
Concision (≤5:30)	15

Final PDF Report (100 pts → 6%)

Criterion	Pts
Abstract & introduction	5
Literature review & differentiators	15
Problem/requirements & stakeholders	15
WhyChain rationale & design trade-offs	15
Architecture diagrams & data model	10
Implementation details (contracts, interface/ops)	10
Evaluation (tests, coverage, gas, ~~usability~~, latency, limits)	15
Ethics & risk	5
AI usage log & reflection	5
Individual contributions & team reflection	5

10) Instructor/TA check-ins (rubric each sprint)

Criterion	Excellent (2)	Developing (1)	Insufficient (0)
Preparedness	Agenda, demo, blockers listed	Partial prep	No prep
Evidence	Live system/tests; updated diagrams	Some evidence	Verbal only
Plan & risks	Clear next steps; risks & mitigations	Vague plan	No plan
Team dynamics	Everyone participates; clear ownership	Uneven participation	Dominated/disengaged

(Scaled to the "professionalism/check-in" points in each sprint.)

11) Submission instructions (Canvas + GitHub)

- One group submission per sprint on Canvas (PDF/links). Each student submits the peer-evaluation form (PQR) each sprint.
- GitHub: push your full project to a team repository; tag each sprint (`sprint1`, `sprint2`, …); add instructor **aazamcs** as a collaborator or share read access; keep the repo public or TA-viewable private.
- For each milestone, also upload to Canvas a ZIP named `67404-FALL25-TEAMNAME-MILESTONE.zip` containing the code snapshot and a short `RUN.md` (setup, commands).

- Metrics Micro-Pack ZIP on Canvas: `67404-TEAMNAME-metrics.zip` containing gas-snapshot.txt (or gas report), coverage-summary.txt (or screenshot), latency.csv, deploy.json, and (Sprint 4) slither-summary.txt.
- Finals: code freeze; upload `67404-FALL25-TEAMNAME-FINAL.zip` (source + docs) to Canvas and ensure the GitHub repo is tagged `final` and accessible to **aazamcs**.
- Video: unlisted link with captions; show repo tag/hash in the recording.
- Naming scheme example: `67404-FALL25-ChainCare-Sprint3.zip`.

12) Checklists & templates

A. WhyChain Canvas (1 page)

- Stakeholders & incentives; trust boundaries; on-chain vs. off-chain split; assets/state & events.
- Threats & abuses (economic, technical, social); alternatives considered; success criteria.

B. Team Charter (≤1 page)

- Roles (rotate at least one role by Sprint 3); decision-making; Definition of Done; meeting cadence; conflict resolution.

C. AI Usage Log (example columns)

- Date; Tool/Model; Prompt/Task; What we kept (and why); Issues found/tests added.

D. Interface Runbook (CLI/Etherscan) — expected contents

• Accounts & funding: which addresses; how to get Sepolia ETH.
• Core flow commands: exact CLI commands (Foundry/Hardhat) or Etherscan URLs/inputs to complete the primary write.
• What to expect: screenshots or copy-pasteable output showing pending→confirmed/failed; where to find tx receipts.
• Troubleshooting: wrong network, insufficient funds, revert messages.

E. Gas Measurement Note (per function)

- Function name & scenario; before/after gas; optimization applied; trade-offs & justification.

13) Practical guidance to avoid setup pain

- Validate in Remix first; move to Foundry/Hardhat once interfaces stabilize.
- Use OpenZeppelin for standard primitives; never commit keys; prefer env vars.
- Pick one testnet; script deployments; tag releases.
- ZK/indexing/account abstraction are optional only after MVP is stable.

14) Have we missed anything?

- Ethics review: include a short "Potential for Harm" note by Sprint 2.
- Backup plan: declare one feature that can be dropped without tanking usability (Sprint 2).
- Role rotation: by Sprint 3, someone new owns tests or UX.
- Reproducibility: clone → install → run tests in <10 minutes on a new machine.

APPENDIX A — Micro-Pack Quick Commands & Templates

These commands are already part of your normal workflow; use them to export the metrics we collect (no extra projects).

A) Foundry commands (preferred)

```
anvil   # run a local test chain

forge test -vv   # run tests with logs

forge coverage   # generate coverage summary/artifacts

forge snapshot   # generate gas snapshot for key functions

forge create --rpc-url $SEPOLIA_RPC_URL \
  --private-key $PRIVATE_KEY \
  src/YourContract.sol:YourContract \
  --verify --verifier etherscan --etherscan-api-key $ETHERSCAN_API_KEY
```

B) Hardhat commands (alternative)

```
npx hardhat coverage   # after installing solidity-coverage

npx hardhat test       # gas reporter prints if configured

npx hardhat verify --network sepolia <DEPLOYED_ADDRESS>
<constructorArgs...>

hardhat.config.* (snippet):
import 'solidity-coverage'
import 'hardhat-gas-reporter'
export default {
  etherscan: { apiKey: process.env.ETHERSCAN_API_KEY },
  gasReporter: { enabled: true },
};
```

C) End-to-end latency/fees helper (TypeScript; paste near your wagmi config as src/txLogger.ts)

```
import { encodeFunctionData, type Abi } from 'viem'
import { writeContract, waitForTransactionReceipt, type
WriteContractParameters } from 'wagmi/actions'
```

```
import { config } from './config'

type TxLogParams = WriteContractParameters & { abi: Abi, functionName:
string, args?: any[] }
export async function runMeasuredWrite(p: TxLogParams) {
  const data = encodeFunctionData({ abi: p.abi, functionName:
p.functionName as any, args: p.args ?? [] })
  const startedAt = Date.now()
  const hash = await writeContract(config, p)
  const receipt = await waitForTransactionReceipt(config, { hash })
  const confirmedAt = Date.now()
  const row = {
    txHash: receipt.transactionHash,
    blockNumber: Number(receipt.blockNumber),
    latencyMs: confirmedAt - startedAt,
    gasUsed: Number(receipt.gasUsed),
    effectiveGasPrice: Number(receipt.effectiveGasPrice),
    calldataBytes: data.length / 2 - 1,
    status: receipt.status
  }
  console.table(row) // copy into latency.csv
  return row
}

Call example:
await runMeasuredWrite({ address: '0xYourSepoliaAddress', abi,
functionName: 'yourFunction', args: [], chainId: 11155111 })
```

D) Micro-Pack ZIP contents (repeated here for convenience)

- gas-snapshot.txt (or Hardhat gas report)
- coverage-summary.txt (or screenshot)
- latency.csv (three runs of your primary write, with fields below)
- deploy.json (chain, address, creation tx, deploy block, compiler, Etherscan link)
- Sprint 4 only: slither-summary.txt

E) CSV/JSON templates (copy these headers)

latency.csv headers:

```
txHash,blockNumber,latencyMs,gasUsed,effectiveGasPrice,calldataBytes,st
atus
```

deploy.json example:

```
{
  "chain": "sepolia",
  "address": "0x...",
  "creationTx": "0x...",
  "deployBlock": 5555555,
```

```
  "compiler": "solc 0.8.24",
  "etherscan": "https://sepolia.etherscan.io/address/0x..."
}
```

usability.csv (for your Usability Test #1)

```
testerId,success,timeOnTaskSec,SEQ_1to7,comment
```

APPENDIX B — ICS calculation & worked examples

ICS formula: ICS = 0.50 × Peer + 0.35 × Evidence + 0.15 × Professionalism (multipliers), capped to [0.50, 1.15].

- Peer multiplier (1–5 → 0.50…1.15): 5→1.15, 4→1.05, 3→0.95, 2→0.75, 1→0.50.
- Evidence multiplier: Strong→1.10, Moderate→1.00, Weak→0.80, Minimal→0.60.
- Professionalism multiplier: Strong→1.05, Moderate→1.00, Weak→0.85.

Student Sprint Grade = Group Score × (0.8 + 0.2 × ICS). Example factors: ICS 0.50→0.90, 0.75→0.95, 0.95→0.99, 1.00→1.00, 1.05→1.01, 1.10→1.02, 1.15→1.03.

Worked examples: see the body rubric section for Sample A–D mapping.

Links to the Project Playbook, Glue Labs, and templates will be posted on Canvas.

Worked Examples (step-by-step)

In every example, **Student Sprint Grade = Group Score × (0.8 + 0.2 × ICS)**.

Example A — Strong contributor (small bonus)
• Group Score = 90
• PeerAvg 4.6 → 1.15; Evidence **Strong** → 1.10; Professionalism **Moderate** → 1.00
• ICS = 0.50×1.15 + 0.35×1.10 + 0.15×1.00
= 0.575 + 0.385 + 0.150 = **1.11**
• Factor = 0.8 + 0.2×1.11 = **1.022**
• **Student Sprint Grade = 90 × 1.022 = 91.98**

Example B — Solid contributor (about the team average)
• Group Score = 88
• PeerAvg 4.0 → 1.05; Evidence **Moderate** → 1.00; Professionalism **Strong** → 1.05
• ICS = 0.5×1.05 + 0.35×1.00 + 0.15×1.05 = **1.0325**
• Factor = 0.8 + 0.2×1.0325 = **1.0065**
• **Student Sprint Grade = 88 × 1.0065 = 88.57**

Example C — Under-contributing (small reduction)
• Group Score = 88
• PeerAvg 3.2 → 0.95; Evidence **Weak** → 0.80; Professionalism **Weak** → 0.85
• ICS = 0.5×0.95 + 0.35×0.80 + 0.15×0.85 = **0.8825**
• Factor = 0.8 + 0.2×0.8825 = **0.9765**
• **Student Sprint Grade = 88 × 0.9765 = 85.93**

Example D — Free-rider territory (single sprint)
• Group Score = 84
• PeerAvg 2.0 → 0.75; Evidence **Minimal** → 0.60; Professionalism **Weak** → 0.85
• ICS = 0.5×0.75 + 0.35×0.60 + 0.15×0.85 = **0.7125**
• Factor = 0.8 + 0.2×0.7125 = **0.9425**
• **Student Sprint Grade = 84 × 0.9425 = 79.17**

If low ICS repeats twice: the instructor may apply the **override penalty** for those sprints:
Student Sprint Grade = 0.8 × Group Score = 0.8 × 84 = 67.20

The End.

Aazam. Last Updated: 20251030.